

DBMS – SALES AND CUSTOMER ANALYTICS SYSTEM

Project Title: EcoCart – Sustainable Products E-Commerce Order Management System

Skill Task: DBMS

Week: 9

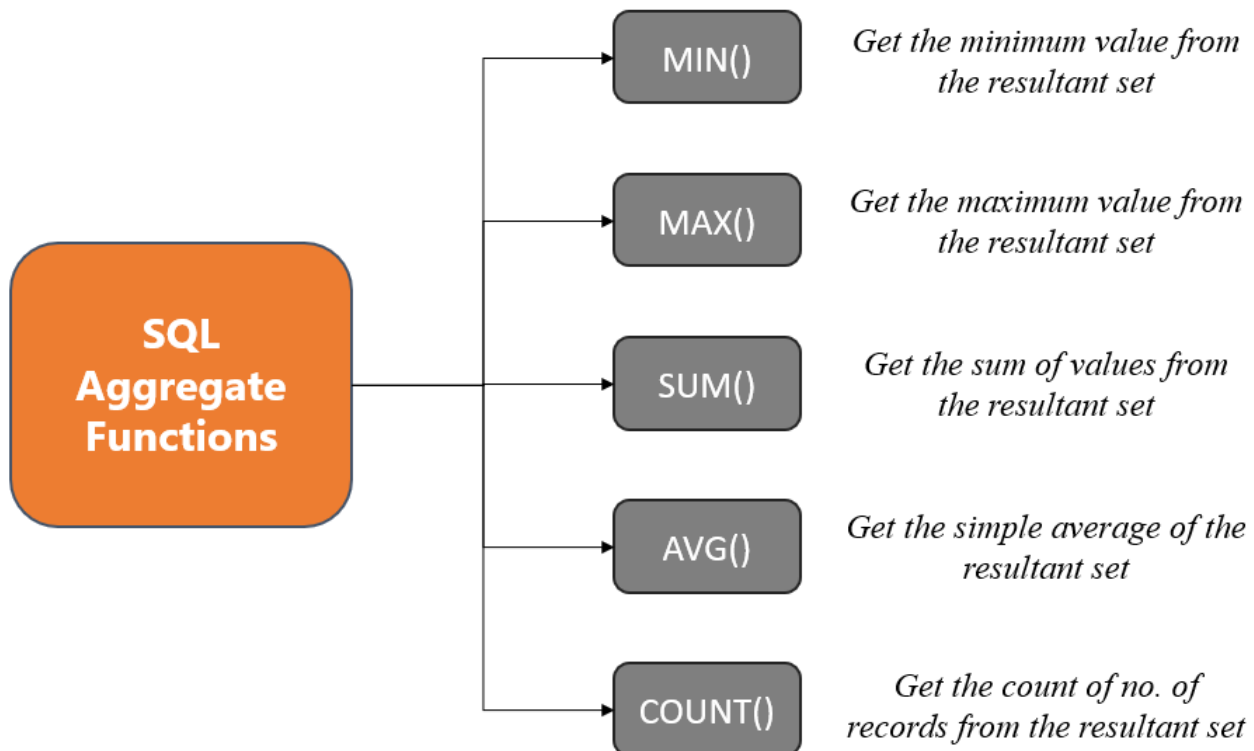
Objectives

1. Apply COUNT(), SUM(), AVG(), MIN(), and MAX().
2. Generate total sales reports.
3. Find top customers based on purchase amount.
4. Identify best-selling products.
5. Perform category-wise sales analysis.

1. Aggregate Functions

Definition

Aggregate functions are SQL functions used to perform calculations on a group of rows and return a single result.



1.1 COUNT()

Find the total number of orders:

```
SQL> SELECT COUNT(Order_ID) AS Total_Orders
      2 FROM Orders;

TOTAL_ORDERS
-----
              4
```

1.2 SUM()

Find the total sales amount:

```
SQL> SELECT SUM(Total_Amount) AS Total_Sales
      2 FROM Orders;

TOTAL_SALES
-----
          9200
```

1.3 AVG()

Find the average order amount:

```
SQL> SELECT AVG(Total_Amount) AS Average_Order_Amount
      2 FROM Orders;

AVERAGE_ORDER_AMOUNT
-----
                  2300
```

1.4 MIN()

Find the lowest order amount:

```
SQL> SELECT MIN(Total_Amount) AS Minimum_Order
      2 FROM Orders;

MINIMUM_ORDER
-----
          1500
```

1.5 MAX()

Find the highest order amount:

```
SQL> SELECT MAX(Total_Amount) AS Maximum_Order
2 FROM Orders;

MAXIMUM_ORDER
-----
3200
```

2. Generate Total Sales Reports

A sales report can be generated using the SUM() function on the Total_Amount column.

```
SQL> SELECT
2 COUNT(Order_ID) AS Total_Orders,
3 SUM(Total_Amount) AS Total_Sales,
4 AVG(Total_Amount) AS Average_Order_Value,
5 MIN(Total_Amount) AS Minimum_Order,
6 MAX(Total_Amount) AS Maximum_Order
7 FROM Orders;

TOTAL_ORDERS TOTAL_SALES AVERAGE_ORDER_VALUE MINIMUM_ORDER MAXIMUM_ORDER
-----
4 9200 2300 1500 3200
```

The output above is based on the sample Orders data used in your EcoCart project.

2.1 Customer-wise Sales Report

To find how much each customer has purchased:

```
SQL> SELECT
2 c.Customer_ID,
3 c.Customer_Name,
4 SUM(o.Total_Amount) AS Total_Purchase
5 FROM Customer c
6 JOIN Orders o
7 ON c.Customer_ID = o.Customer_ID
8 GROUP BY
9 c.Customer_ID,
10 c.Customer_Name
11 ORDER BY Total_Purchase DESC;

CUSTOMER_ID
-----
CUSTOMER_NAME
-----
TOTAL_PURCHASE
-----
103
Priya 3200
101
Anitha Kumar 2700
```

Purpose

This report shows:

- Customer ID
- Customer name
- Total purchase amount
- Customers arranged according to purchase amount

3. Find Top Customers Based on Purchase Amount

The SUM() function can be combined with GROUP BY and ORDER BY to identify customers with higher purchase amounts.

```
SQL> SELECT
  2     c.Customer_ID,
  3     c.Customer_Name,
  4     SUM(o.Total_Amount) AS Total_Purchase
  5 FROM Customer c
  6 JOIN Orders o
  7   ON c.Customer_ID = o.Customer_ID
  8 GROUP BY
  9     c.Customer_ID,
 10     c.Customer_Name
 11 ORDER BY Total_Purchase DESC;
```

```
CUSTOMER_ID
-----
CUSTOMER_NAME
-----
TOTAL_PURCHASE
-----
      103
Priya      3200
      101
Anitha Kumar 2700
CUSTOMER_ID
-----
CUSTOMER_NAME
```

The customer appearing at the top has the highest purchase amount in the generated report.

4. Identify Best-Selling Products

The Order_Details table contains the quantity of each product ordered.

Therefore, we can use SUM(Quantity) to find the total quantity sold for each product.

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     SUM(od.Quantity) AS Total_Quantity_Sold
  5 FROM Product p
  6 JOIN Order_Details od
  7     ON p.Product_ID = od.Product_ID
  8 GROUP BY
  9     p.Product_ID,
 10     p.Product_Name
 11 ORDER BY Total_Quantity_Sold DESC;
```

PRODUCT_ID

PRODUCT_NAME

TOTAL_QUANTITY_SOLD

101	
Organic Turmeric Powder	4

103	
Organic Honey	2

Purpose

This query:

1. Connects Product and Order_Details.
2. Groups records according to product.
3. Calculates total quantity sold.
4. Displays products from highest to lowest quantity sold.

4.1 Product Sales Amount

Instead of only calculating quantity, we can calculate the total sales generated by each product.

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     SUM(od.Quantity) AS Total_Quantity_Sold,
  5     SUM(od.Subtotal) AS Total_Sales
  6 FROM Product p
  7 JOIN Order_Details od
  8     ON p.Product_ID = od.Product_ID
  9 GROUP BY
 10     p.Product_ID,
 11     p.Product_Name
 12 ORDER BY Total_Sales DESC;
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
TOTAL_QUANTITY_SOLD TOTAL_SALES
-----
      101
Organic Turmeric Powder
              4      4500

      104
Bamboo Storage Box
              2      3200
```

This gives both **quantity sold** and **sales amount**.

5. Category-wise Sales Analysis

Category-wise analysis helps determine the sales generated by each product category.

```
SQL> SELECT
  2     c.Category_ID,
  3     c.Category_Name,
  4     SUM(od.Quantity) AS Total_Quantity_Sold,
  5     SUM(od.Subtotal) AS Total_Sales
  6 FROM Category c
  7 JOIN Product p
  8     ON c.Category_ID = p.Category_ID
  9 JOIN Order_Details od
 10     ON p.Product_ID = od.Product_ID
 11 GROUP BY
 12     c.Category_ID,
 13     c.Category_Name
 14 ORDER BY Total_Sales DESC;
```

```
CATEGORY_ID
-----
CATEGORY_NAME
-----
TOTAL_QUANTITY_SOLD TOTAL_SALES
-----
      1
Spices
              5      5000

      3
Eco Friendly Home
              2      3200

CATEGORY_ID
```

6. Combined Business Sales Report

A complete sales report can combine customer, order, product, and category information.

```
SQL> SELECT
2      c.Customer_Name,
3      o.Order_ID,
4      p.Product_Name,
5      cat.Category_Name,
6      od.Quantity,
7      od.Subtotal
8 FROM Customer c
9 JOIN Orders o
10      ON c.Customer_ID = o.Customer_ID
11 JOIN Order_Details od
12      ON o.Order_ID = od.Order_ID
13 JOIN Product p
14      ON od.Product_ID = p.Product_ID
15 JOIN Category cat
16      ON p.Category_ID = cat.Category_ID
17 ORDER BY o.Order_ID;
```

CUSTOMER_NAME	
ORDER_ID	
PRODUCT_NAME	
CATEGORY_NAME	
QUANTITY	SUBTOTAL

Report Structure

This report can be used to understand:

- Who purchased the product
- Which order was placed
- Which product was purchased
- Which category the product belongs to
- Quantity purchased
- Amount generated